LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

M Al Kahfi Hafidz M 2609106013

Kelas A1 ‘26

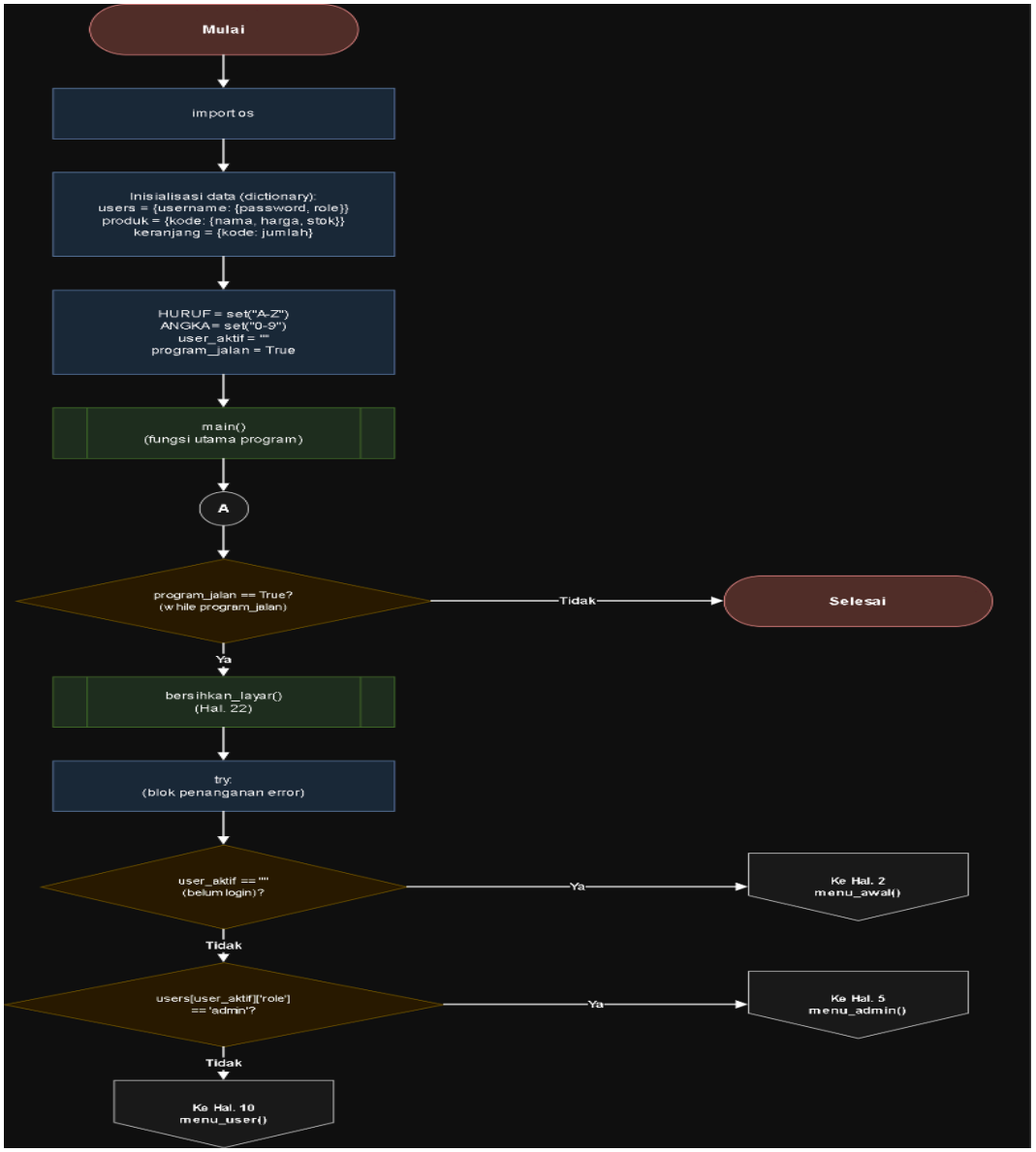
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

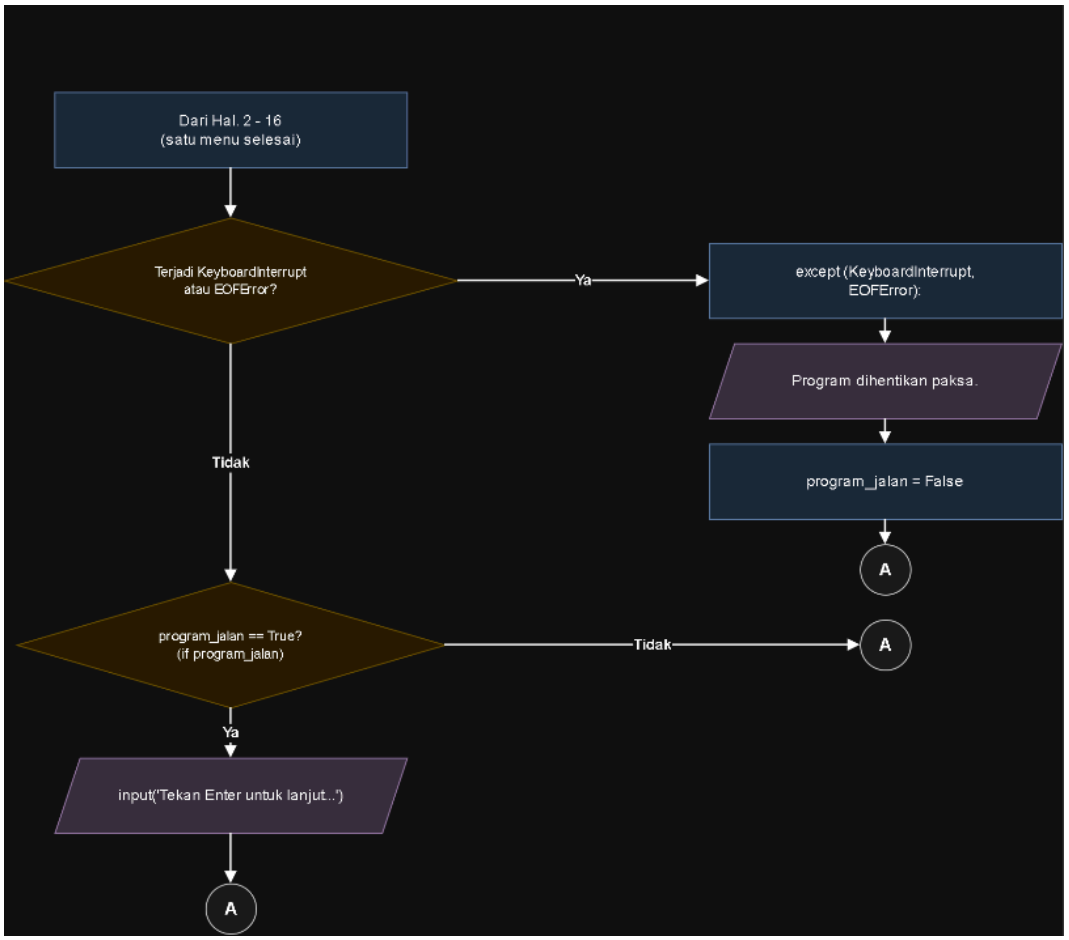
SAMARINDA

2026

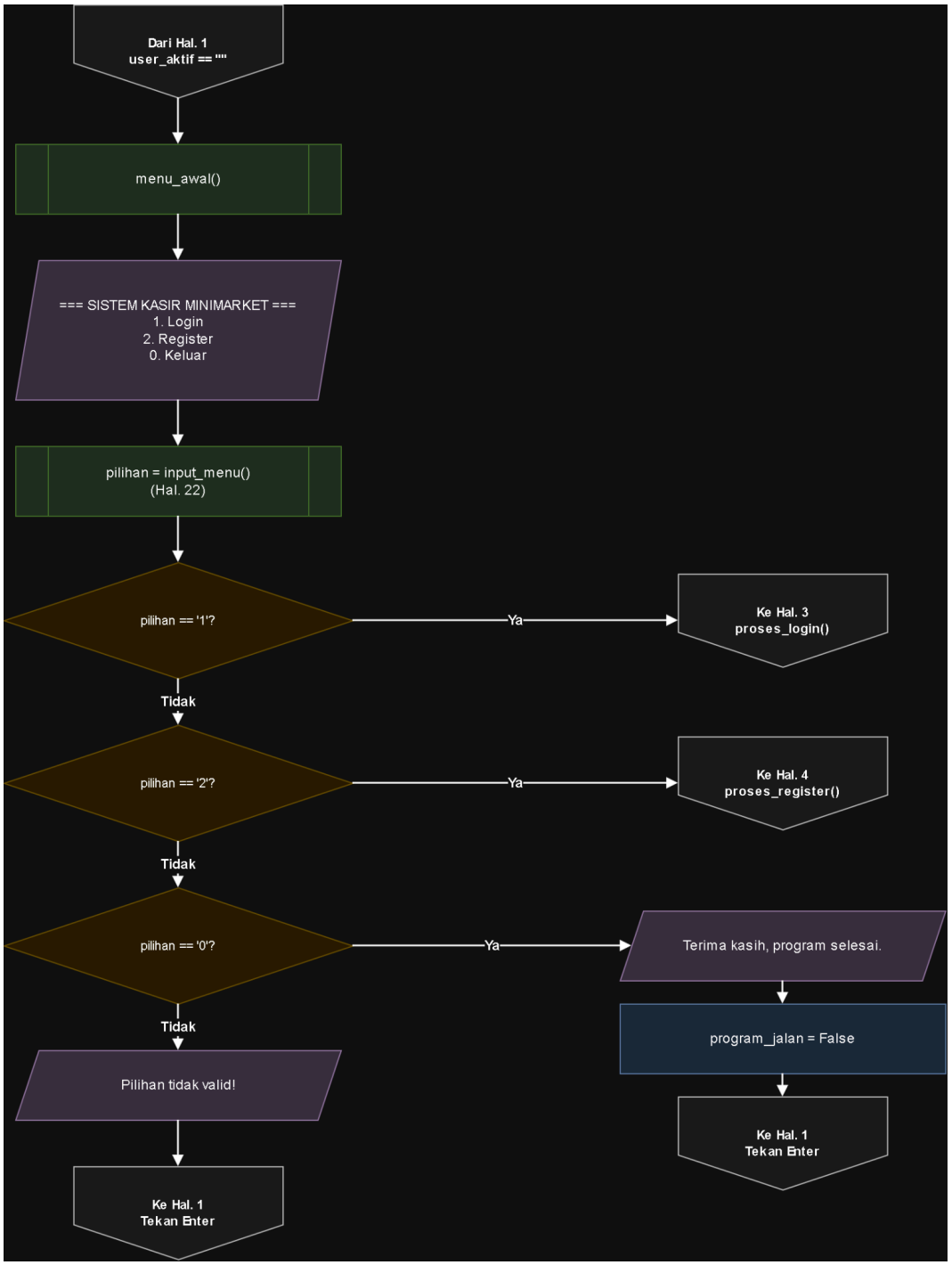
1. Flowchart



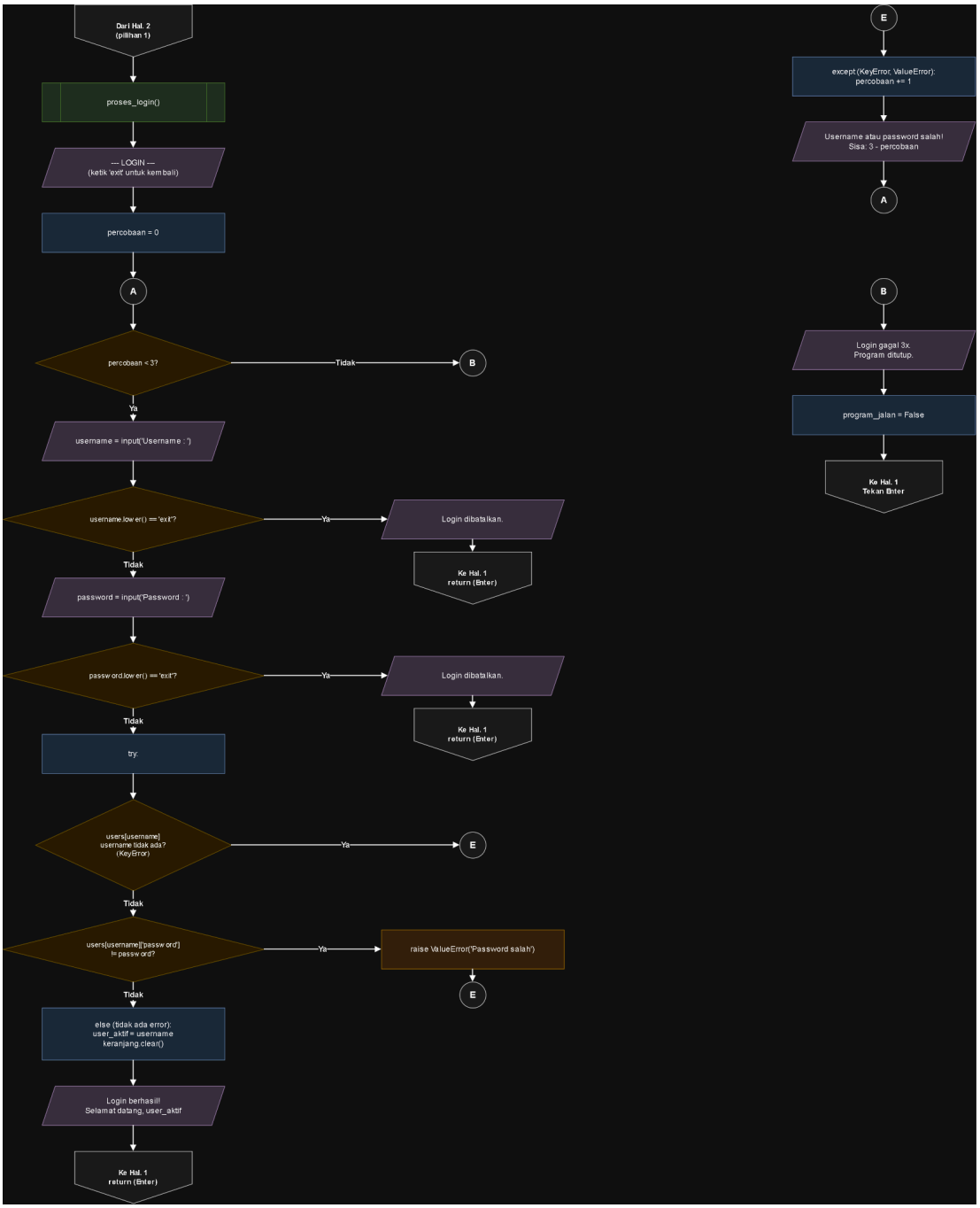
Gambar 1.1 *Flowchart Program Utama (Loop)*



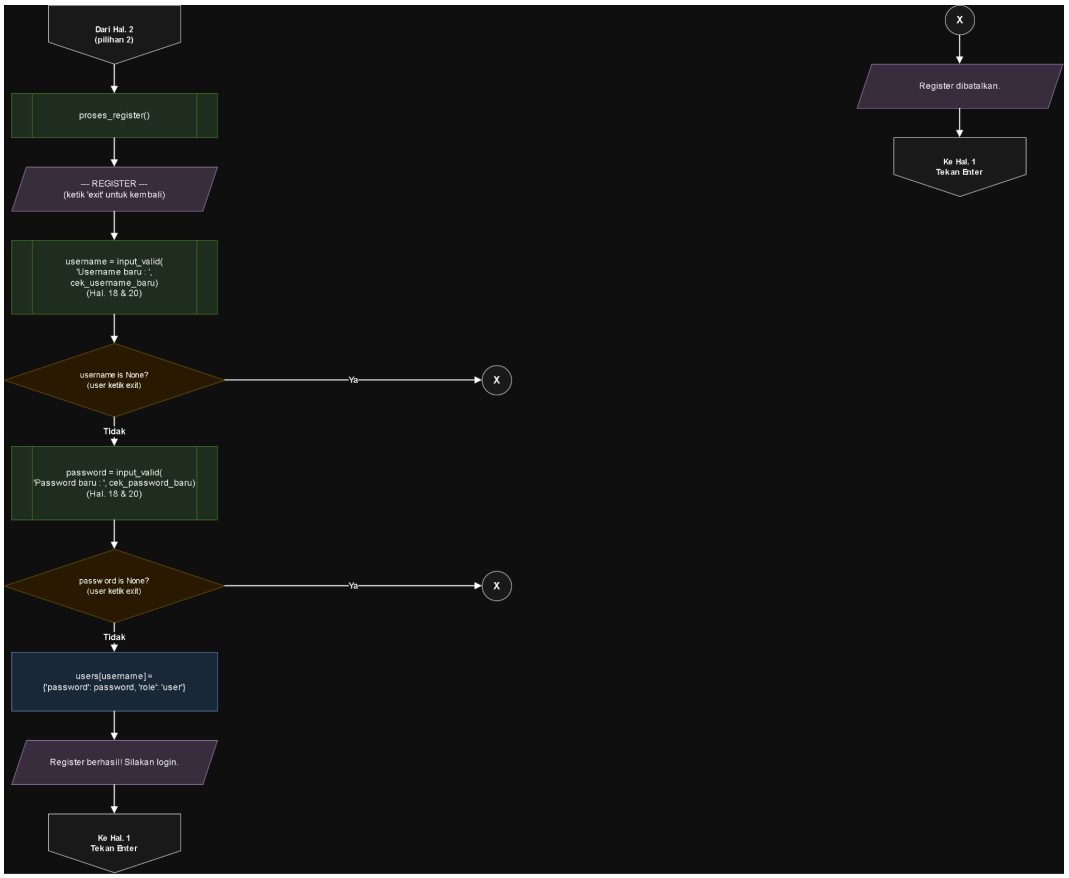
Gambar 1.2 *Flowchart Program Utama (Loop) 2*



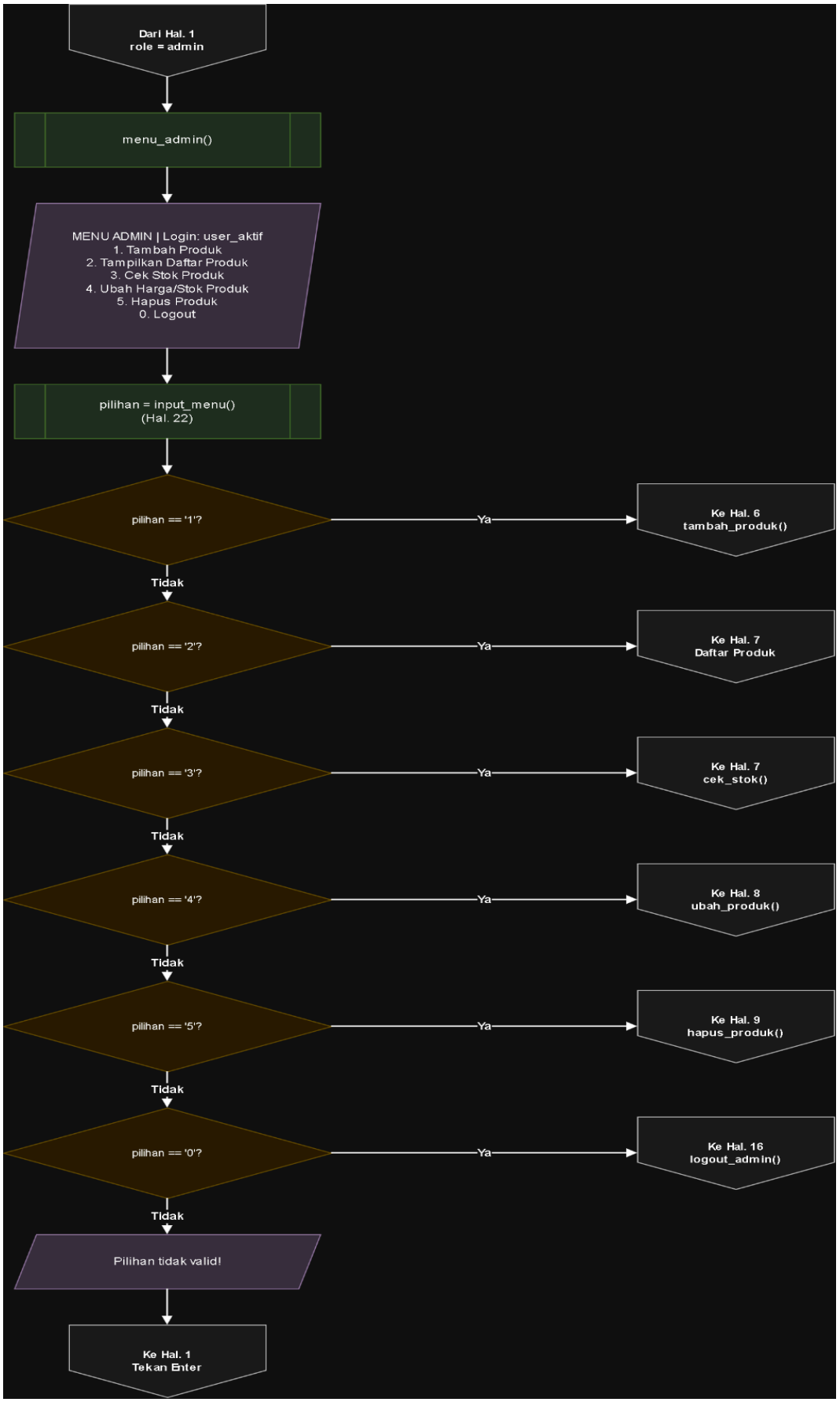
Gambar 1.3 *Flowchart Menu awal*



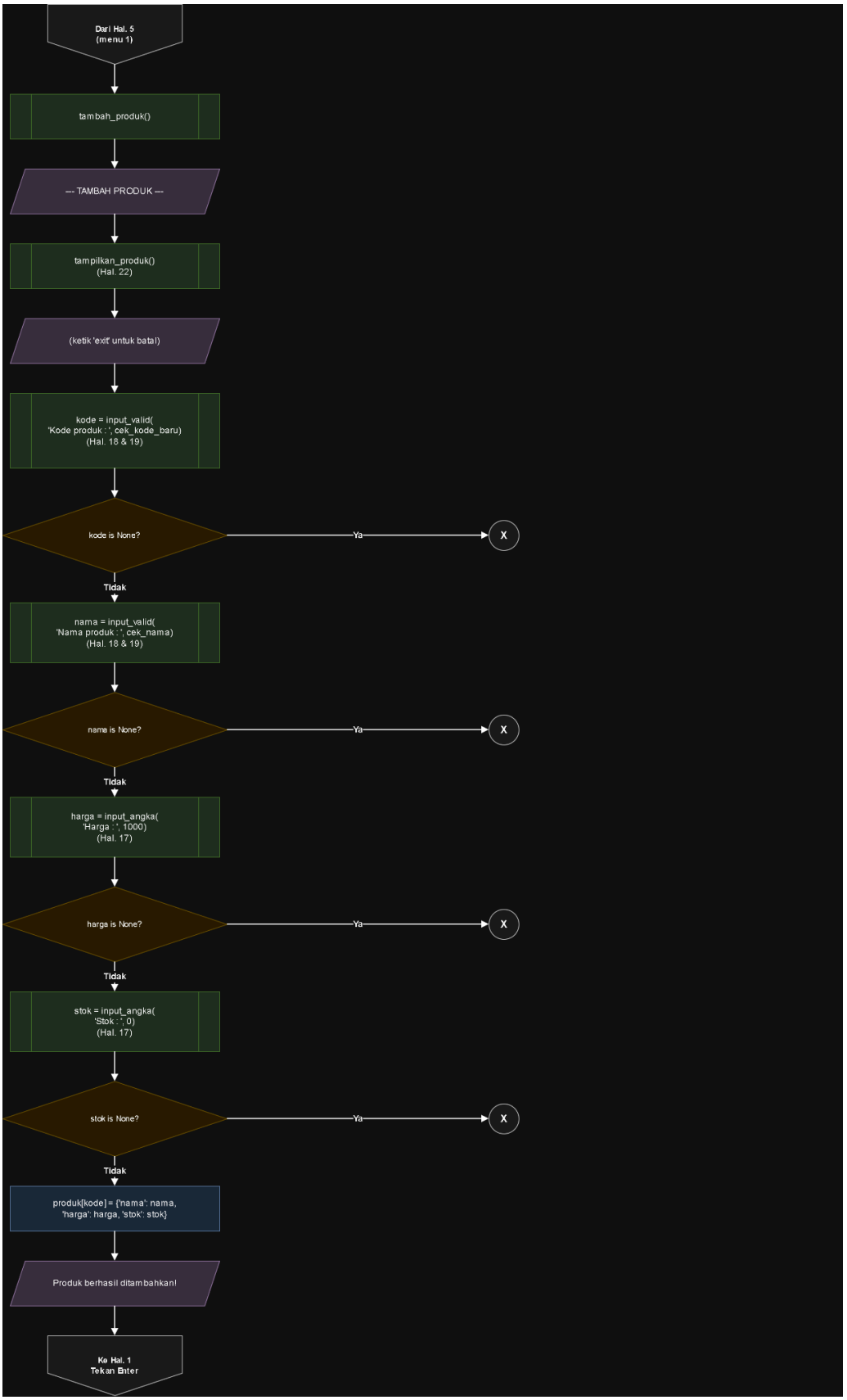
Gambar 1.4 *Flowchart Login*



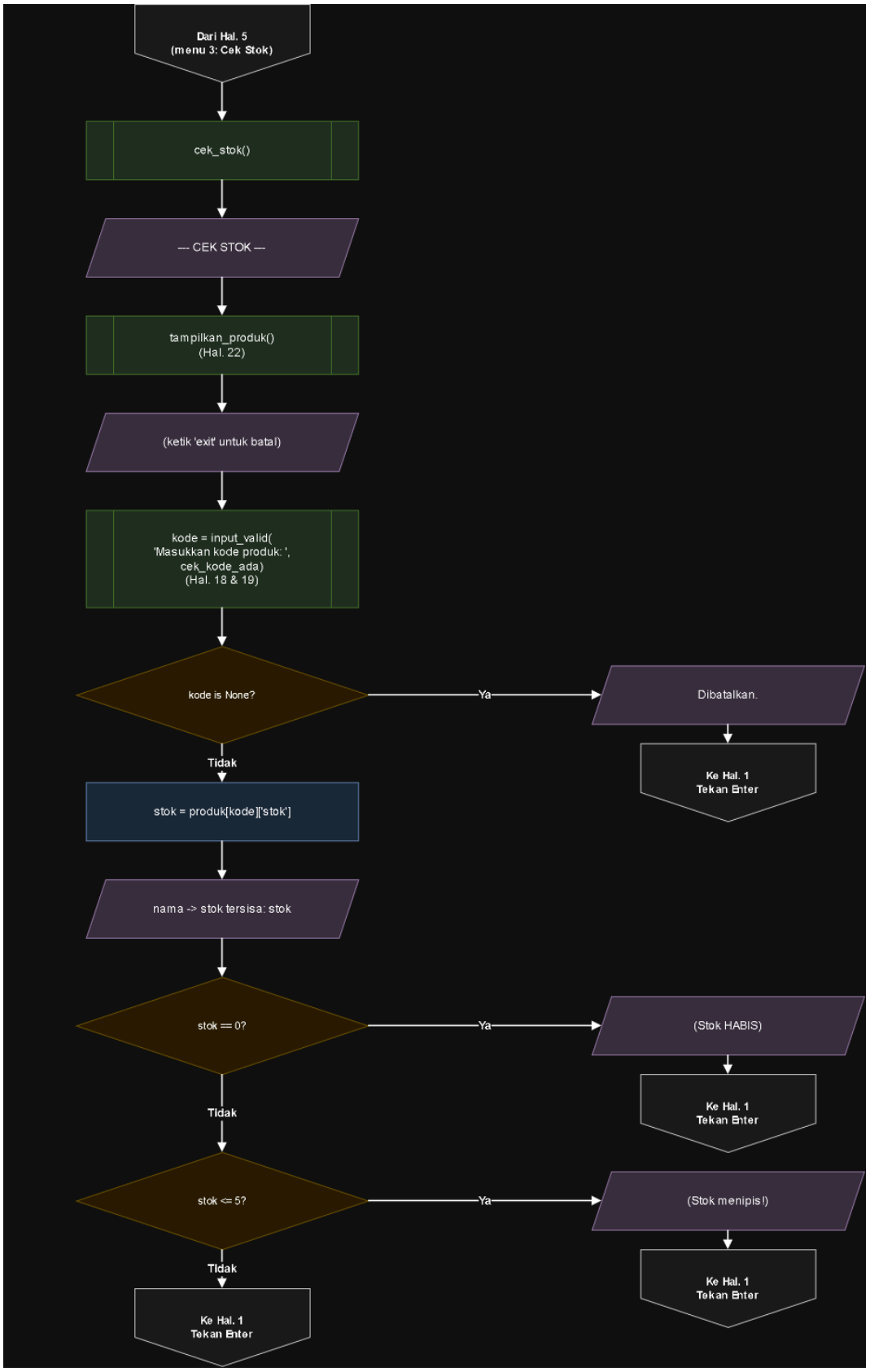
Gambar 1.5 *Flowchart Register*



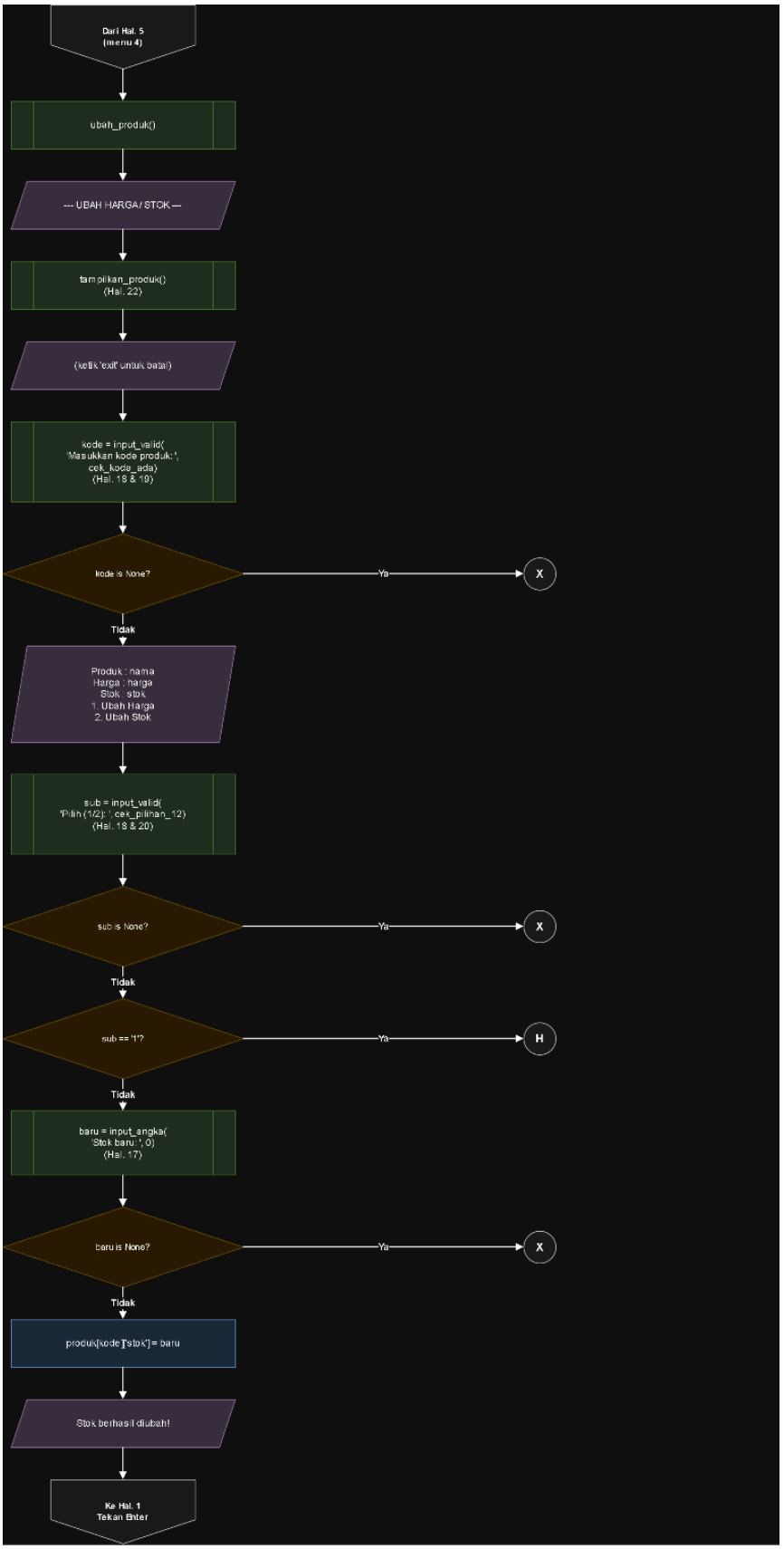
Gambar 1.6 *Flowchart Menu Admin*



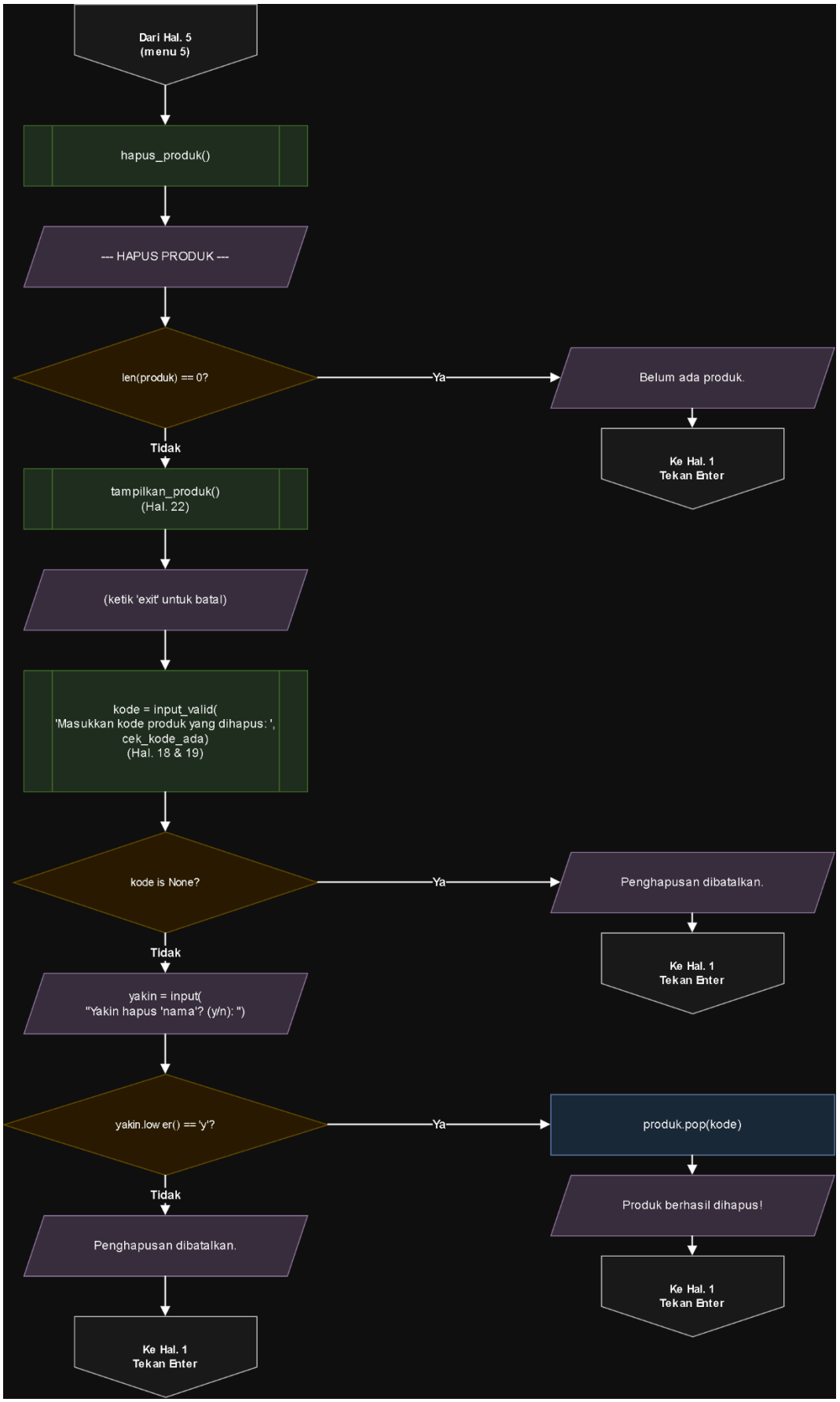
Gambar 1.7 *Flowchart Tambah Produk*



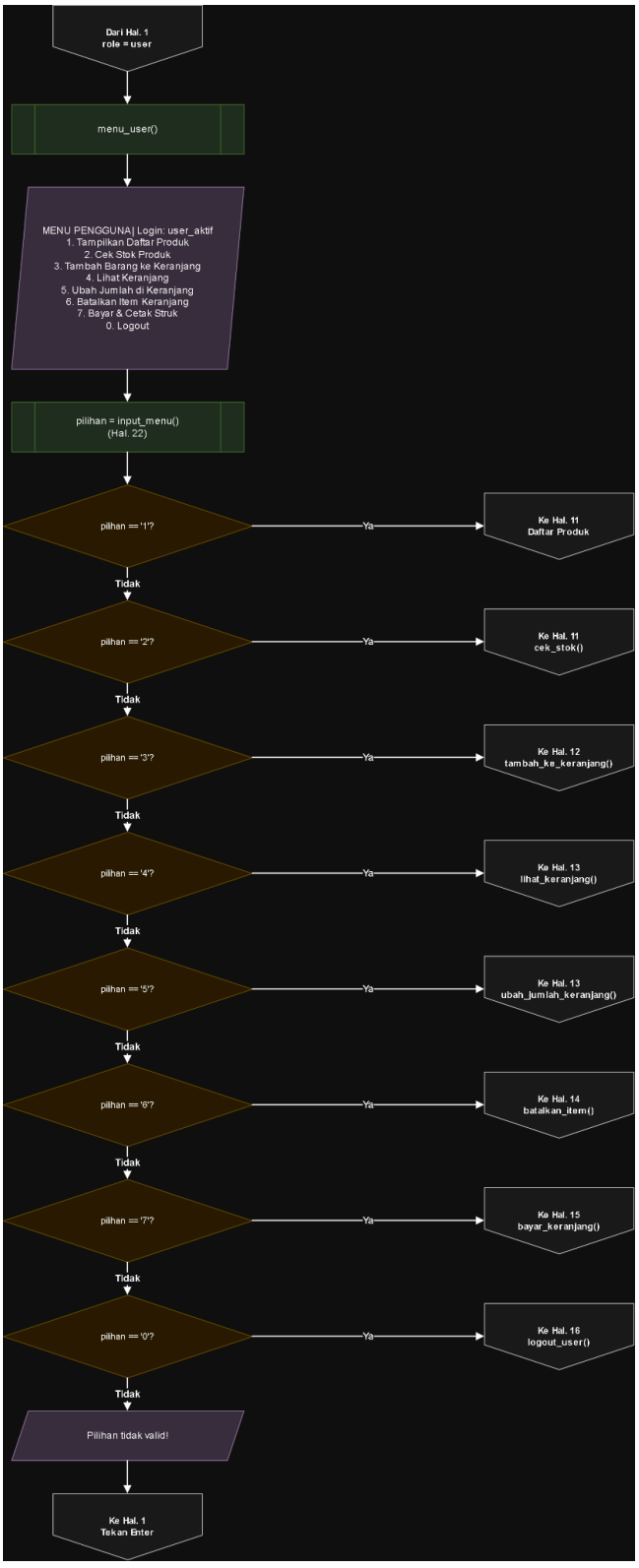
Gambar 1.8 *Flowchart Admin Daftar dan Cek Stok*



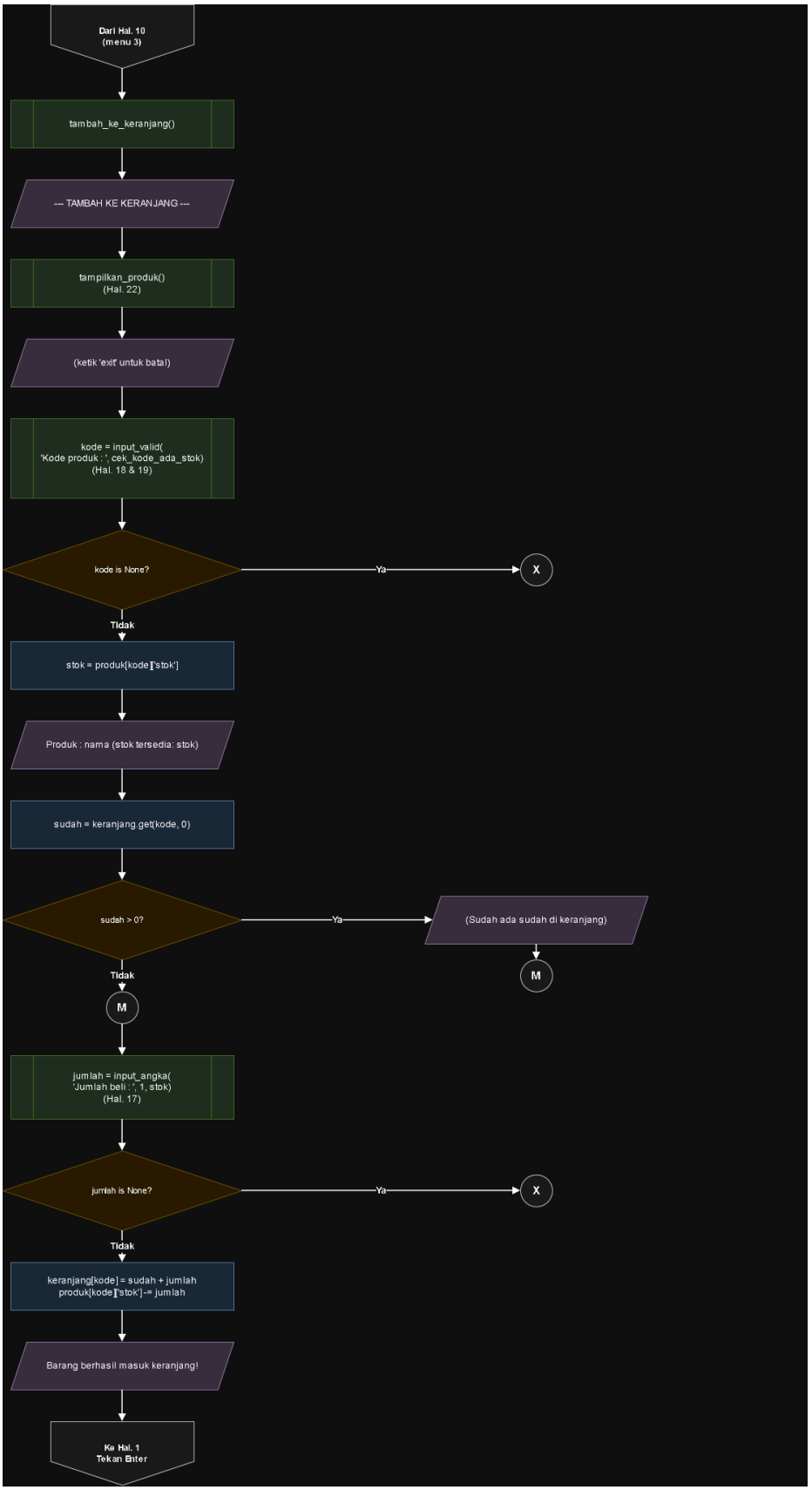
Gambar 1.9 *Flowchar Admin Ubah Harga*



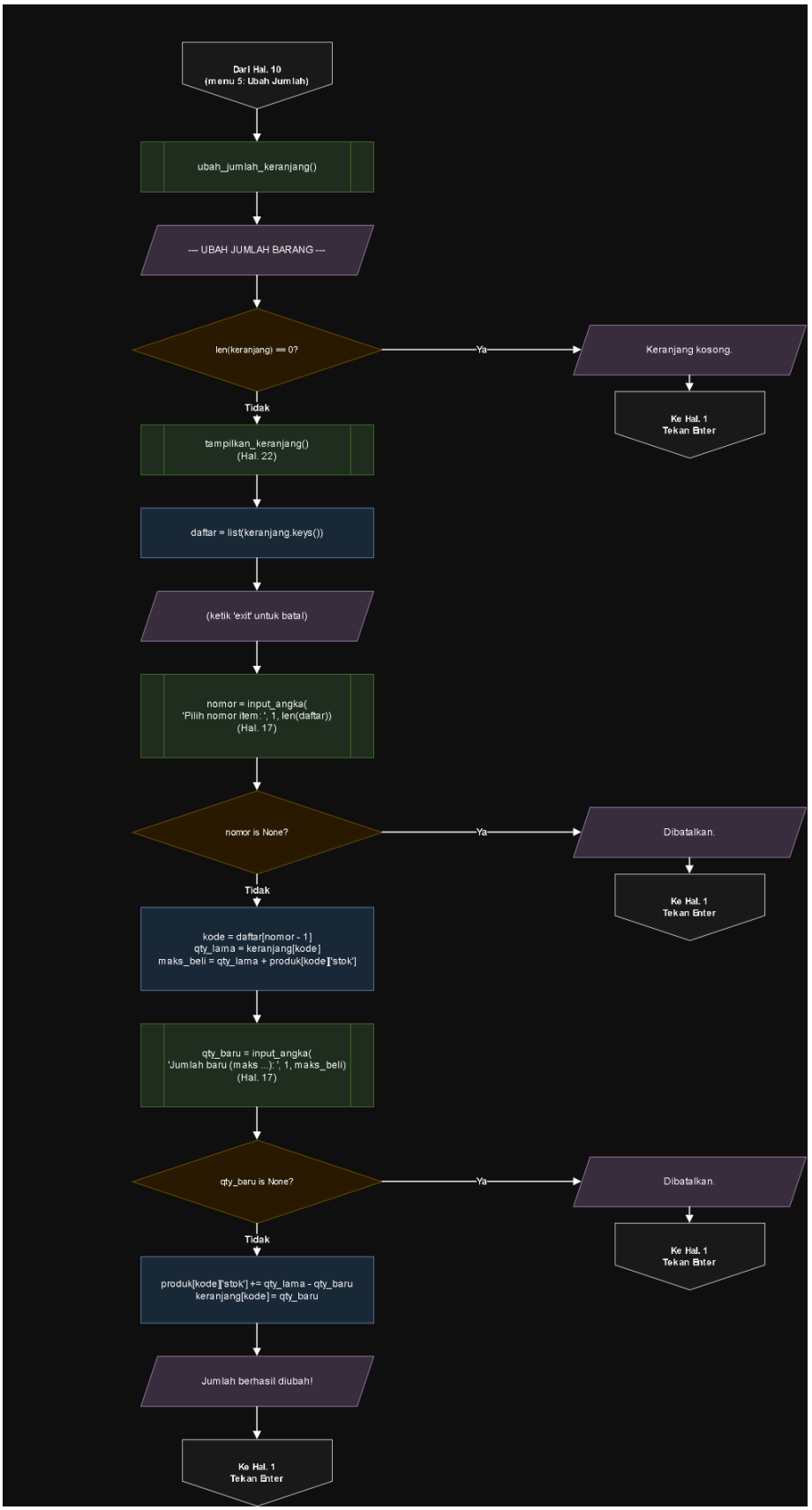
Gambar 1.10 *Flowchart Admin Hapus Produk*



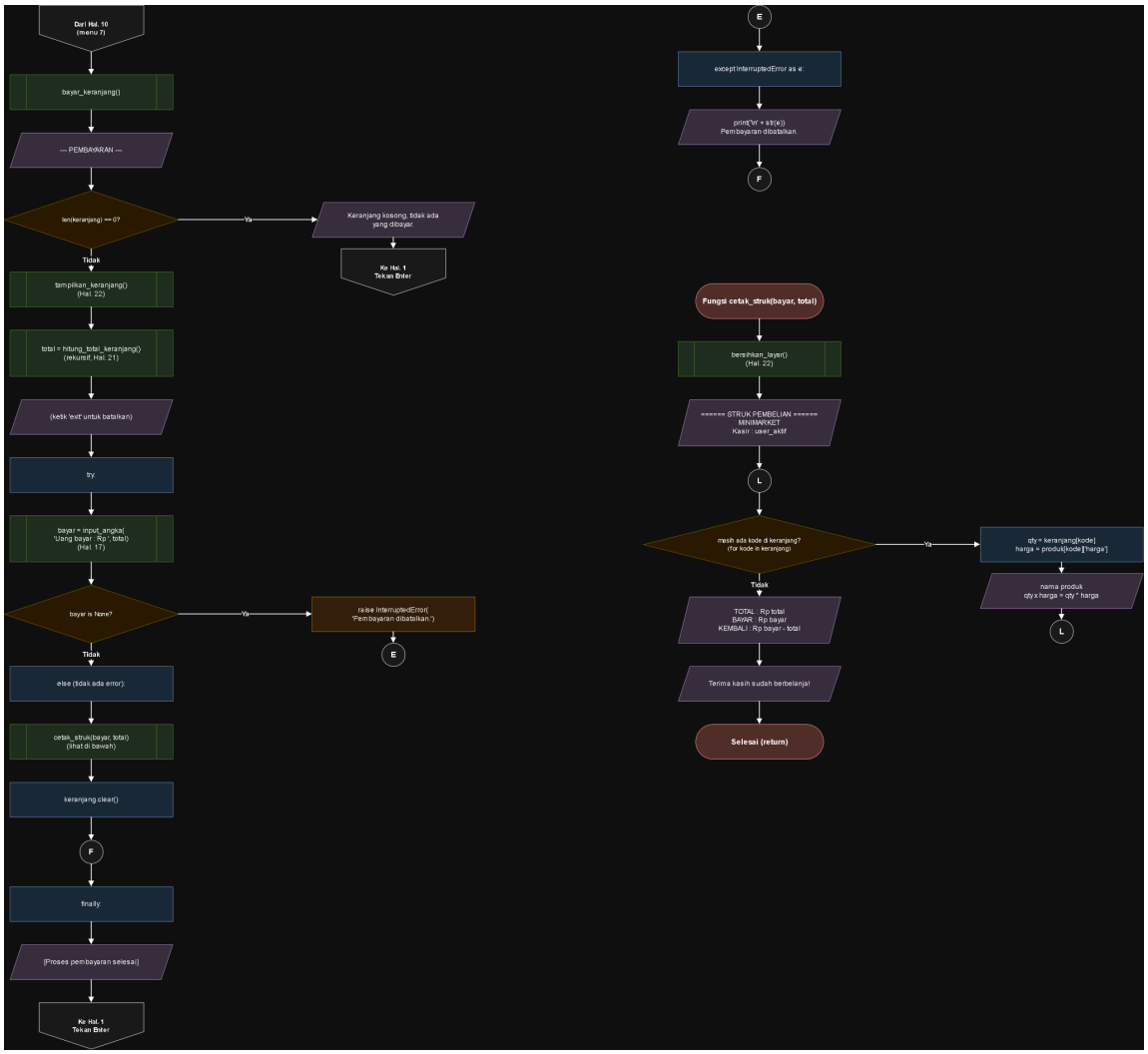
Gambar 1.11 *Flowchart Menu Pengguna*



Gambar 1.12 *Flowchart User Tambahkan Keranjang*



Gambar 1.13 *Flowchart User Lihat dan Ubah Keranjang*



Gambar 1.14 *Flowchart User Bayar dan Struk*

2. Deskripsi Singkat Program

Program ini adalah sistem kasir minimarket berbasis terminal yang dibuat dengan Python. Pada Posttest 6 ini, penyimpanan data diubah menggunakan struktur data Dictionary (dict) untuk menyimpan akun users, daftar produk, dan isi keranjang, sehingga pengelolaan data lebih efisien. Semua data disimpan di memori dan akan hilang saat program ditutup.

Saat dijalankan, pengguna bisa memilih Login, Register, atau Keluar. Login dibatasi 3 kali percobaan. Register memvalidasi username (tidak kosong, tanpa spasi, belum dipakai) serta password minimal 4 karakter. Setelah login, menu yang muncul tergantung peran pengguna:

- **Admin** mengelola data produk: menambah produk dengan validasi kombinasi huruf dan angka, menampilkan daftar, mengecek stok, mengubah harga atau stok, dan menghapus produk.
- **Kasir/Pengguna** melayani pembelian: melihat produk, menambah barang ke keranjang berdasarkan stok, mengubah jumlah, membatalkan item, serta melakukan pembayaran dan mencetak struk.

3. Source Code

A. Inisialisasi Data Dictionary

Data pengguna, produk, dan keranjang disimpan dalam dictionary.

```
import os

users = {
    "admin": {"password": "admin17", "role": "admin"},
    "kasir": {"password": "kasir17", "role": "user"},
}

produk = {
    "P001": {"nama": "Indomie Goreng", "harga": 3500, "stok": 50},
    # ... (produk lainnya)
}

keranjang = {}
user_aktif = ""
program_jalan = True
```

B. Fungsi Rekursif

Fungsi rekursif dipakai untuk menjumlahkan subtotal seluruh item di keranjang, dan untuk mengulang input angka sampai valid.

```
def hitung_total_rekursif(daftar_kode):
    if len(daftar_kode) == 0:  # basis
        return 0
    return hitung_subtotal(daftar_kode[0]) +
hitung_total_rekursif(daftar_kode[1:])


def input_angka(pesan, minimal=0, maksimal=None):
    teks = input(pesan).strip()
    if teks.lower() == "exit":
        return None
    try:
        nilai = int(teks)
        if nilai < minimal:
            raise ValueError(f"Nilai minimal {minimal}!")
        if maksimal is not None and nilai > maksimal:
            raise ValueError(f"Nilai maksimal {maksimal}!")
    except ValueError as e:
        print("Error:", e)
        return input_angka(pesan, minimal, maksimal)  # rekursif
    else:
        return nilai
```

C. Error Handling & Validasi Input

Validator melempar ValueError bila input salah, lalu input_valid menangkapnya dan meminta input ulang secara rekursif.

```
def input_valid(pesan, validator):
    teks = input(pesan).strip()
    if teks.lower() == "exit":
        return None
    try:
        hasil = validator(teks)
    except ValueError as e:
        print("Error:", e)
        return input_valid(pesan, validator)
```

```
    else:
        return hasil


def cek_kode_baru(kode):
    kode = kode.upper()
    if kode == "":
        raise ValueError("Kode tidak boleh kosong!")
    if len(set(kode) & HURUF) == 0 or len(set(kode) & ANGKA) == 0:
        raise ValueError("Kode harus gabungan huruf dan angka!")
    if kode in produk:
        raise ValueError("Kode produk sudah ada!")
    return kode
```

D. Fitur Login

Memvalidasi username dan password pada dictionary users. Pengguna diberi 3 kali kesempatan login.

```
def proses_login():
    global user_aktif, program_jalan
    percobaan = 0
    while percobaan < 3:
        username = input("Username : ")
        password = input("Password : ")
        try:
            if users[username]["password"] != password:
                raise ValueError("Password salah")
        except (KeyError, ValueError):
            percobaan += 1
            print("Username atau password salah! Sisa:", 3 - percobaan)
        else:
            user_aktif = username
            keranjang.clear()
            print("Login berhasil! Selamat datang,", user_aktif)
            return
    print("Login gagal 3x. Program ditutup.")
    program_jalan = False
```

E. Fitur Register

Mendaftarkan akun baru ke dictionary users dengan role user.

```
def proses_register():
    username = input_valid("Username baru : ", cek_username_baru)
    password = input_valid("Password baru : ", cek_password_baru)
    users[username] = {"password": password, "role": "user"}
    print("Register berhasil! Silakan login.")
```

F. Fitur Tambah Produk (Admin)

Menyimpan produk baru ke dictionary produk setelah semua input tervalidasi.

```
def tambah_produk():
    kode = input_valid("Kode produk : ", cek_kode_baru)
    nama = input_valid("Nama produk : ", cek_nama)
    harga = input_angka("Harga       : ", 1000)
    stok = input_angka("Stok        : ", 0)
    produk[kode] = {"nama": nama, "harga": harga, "stok": stok}
    print("\nProduk berhasil ditambahkan!")
```

G. Fitur Tambah Barang ke Keranjang (Kasir)

Mengecek stok pada dictionary produk, lalu menyimpan jumlah beli ke dictionary keranjang dengan key kode barang.

```
def tambah_ke_keranjang():
    kode = input_valid("Kode produk : ", cek_kode_ada_stok)
    stok = produk[kode]["stok"]
    sudah = keranjang.get(kode, 0)
    jumlah = input_angka("Jumlah beli : ", 1, stok)
    keranjang[kode] = sudah + jumlah
    produk[kode]["stok"] -= jumlah
    print("\nBarang berhasil masuk keranjang!")
```

H. Fitur Pembayaran (Kasir)

Memakai try-except-else-finally: pembayaran dibatalkan jika user mengetik exit, dan struk dicetak jika berhasil.

```
def bayar_keranjang():
    total = hitung_total_keranjang()
    try:
        bayar = input_angka("Uang bayar : Rp ", total)
        if bayar is None:
            raise InterruptedError("Pembayaran dibatalkan.")
    except InterruptedError as e:
        print("\n" + str(e))
    else:
        cetak_struk(bayar, total)
        keranjang.clear()
    finally:
        print("\n[Proses pembayaran selesai]")
```

I. Program Utama (Loop)

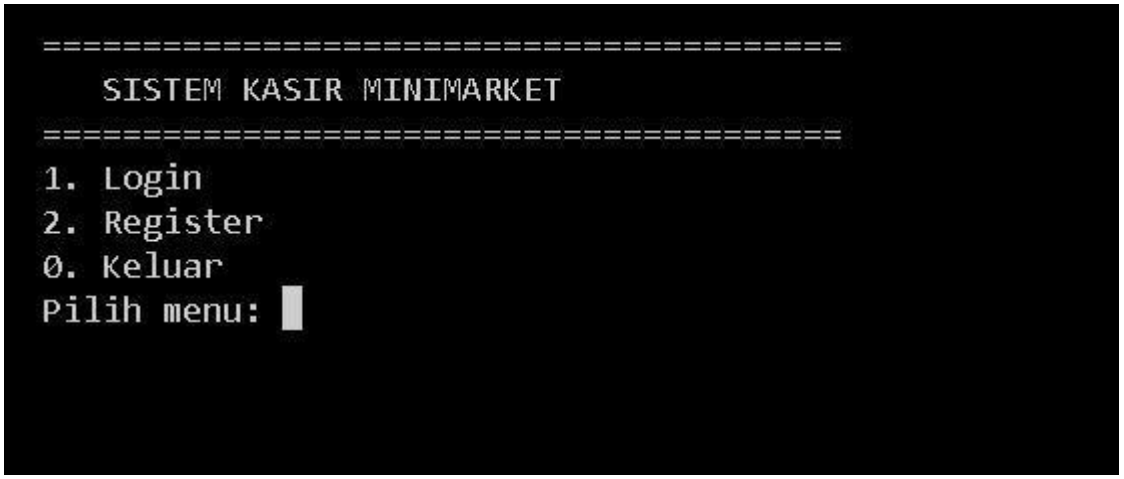
Loop utama menampilkan menu sesuai status login dan role, serta menangani penghentian paksa.

```
def main():
    global program_jalan
    while program_jalan:
        bersihkan_layar()
        try:
            if user_aktif == "":
                menu_awal()
            elif users[user_aktif]["role"] == "admin":
                menu_admin()
            else:
                menu_user()
            if program_jalan:
                input("\nTekan Enter untuk lanjut...")
        except (KeyboardInterrupt, EOFError):
            print("\nProgram dihentikan paksa.")
            program_jalan = False


main()
```

4. Hasil Output

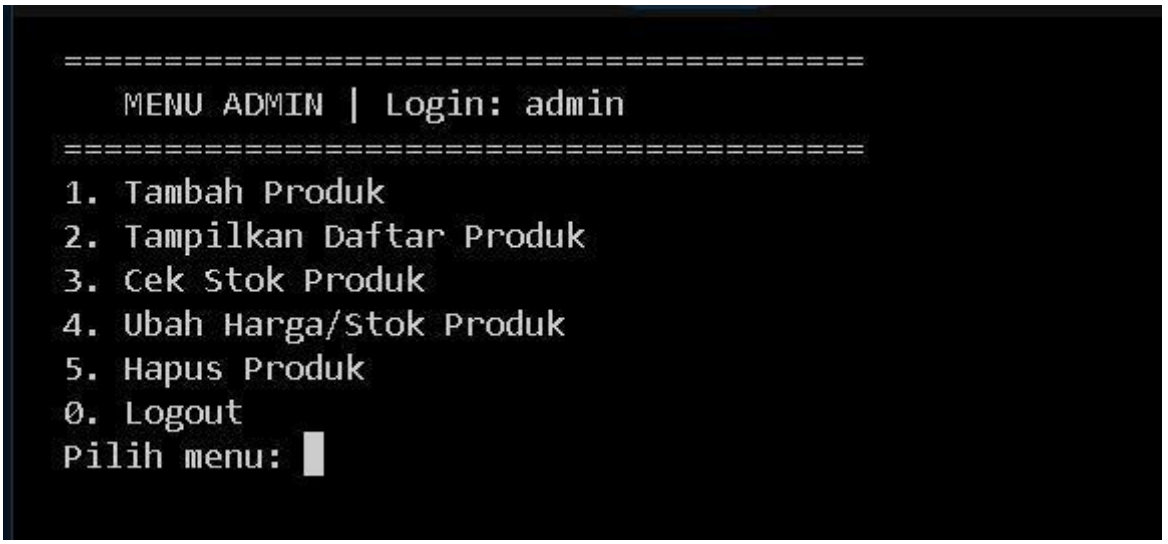
4. Hasil Output



Gambar 4.1 Tampilan Login, Register dan Keluar



Gambar 4.2 Login Admin



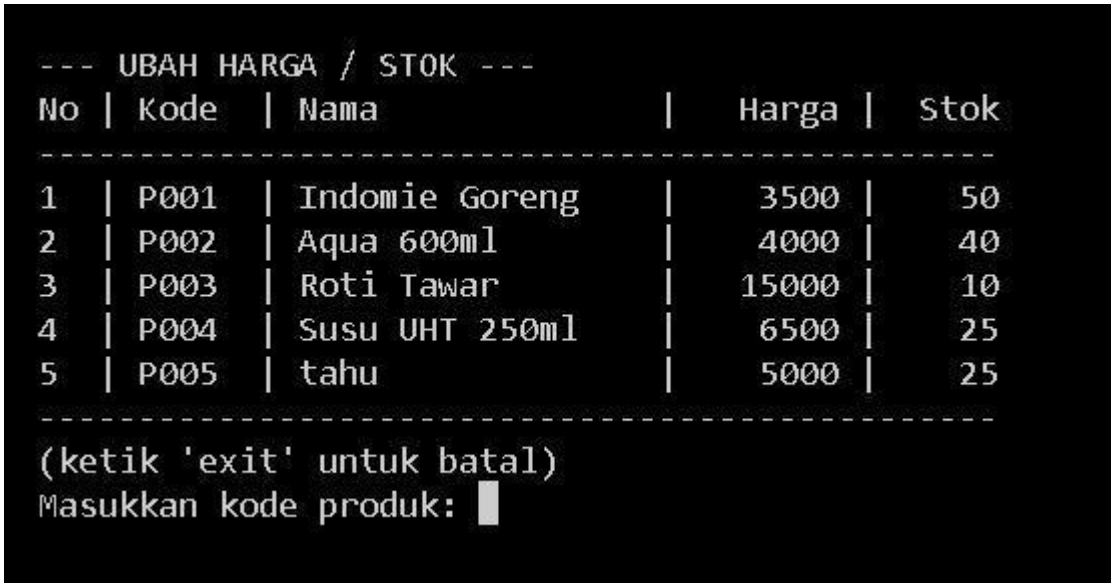
Gambar 4.3 Tampilan Menu Admin



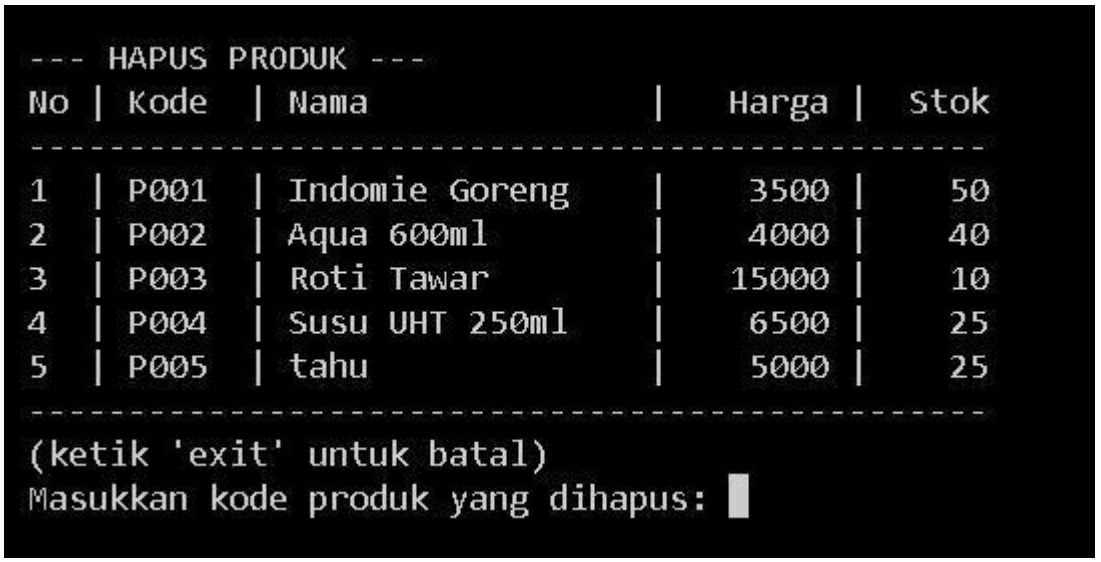
Gambar 4.4 Tampilan Daftar Produk



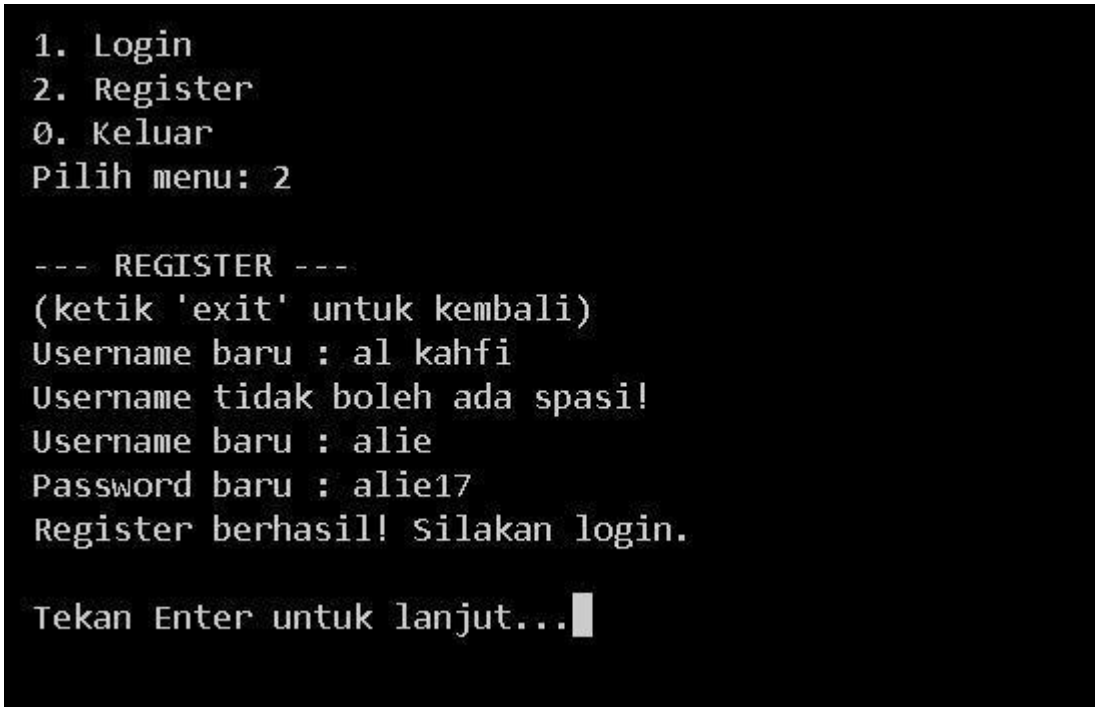
Gambar 4.5 Tampilan Cek Stok



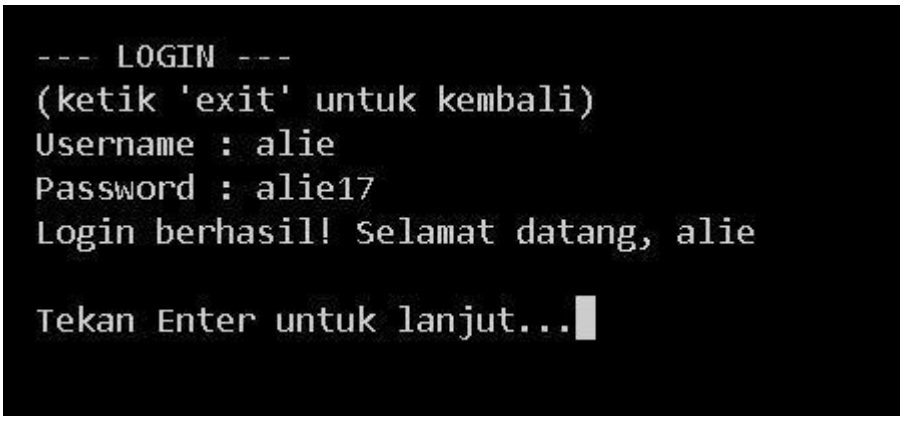
Gambar 4.6 Tampilan Ubah Harga/Stok



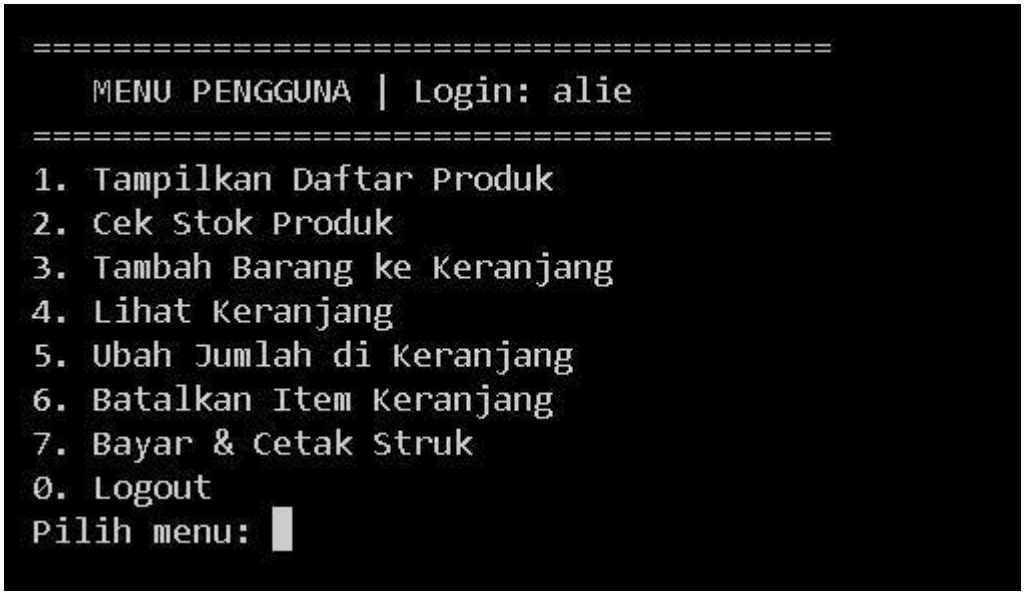
Gambar 4.7 Tampilan Hapus Produk



Gambar 4.8 Tampilan Menu Register



Gambar 4.9 Tampilan Tampilan Login User



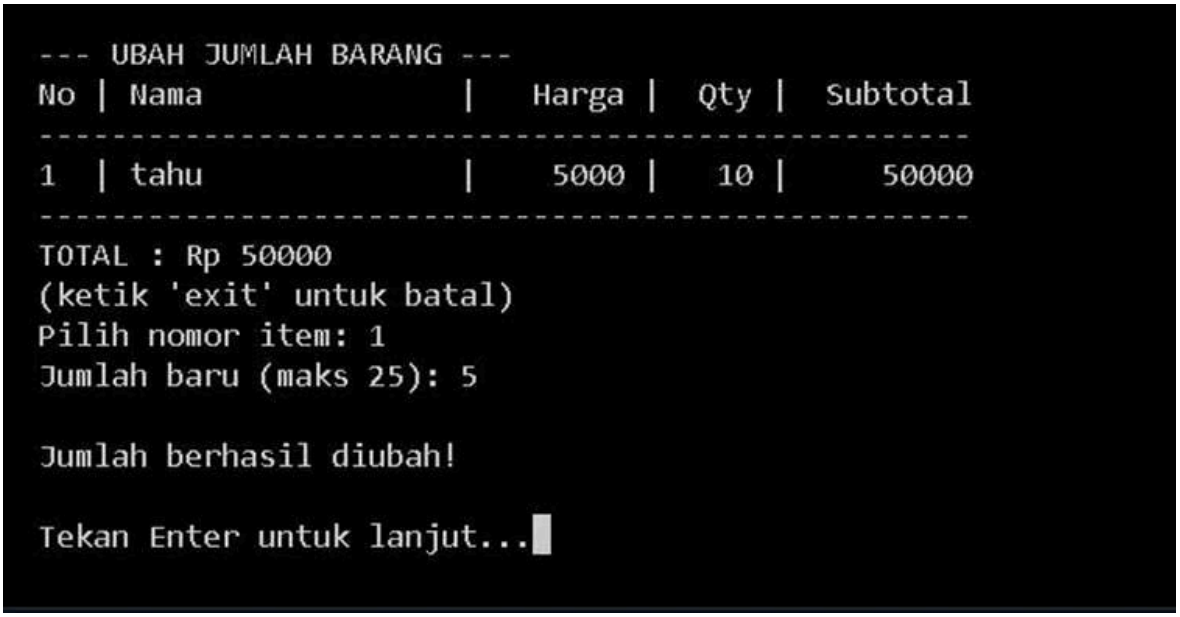
Gambar 4.10 Tampilan Tampilan Menu User



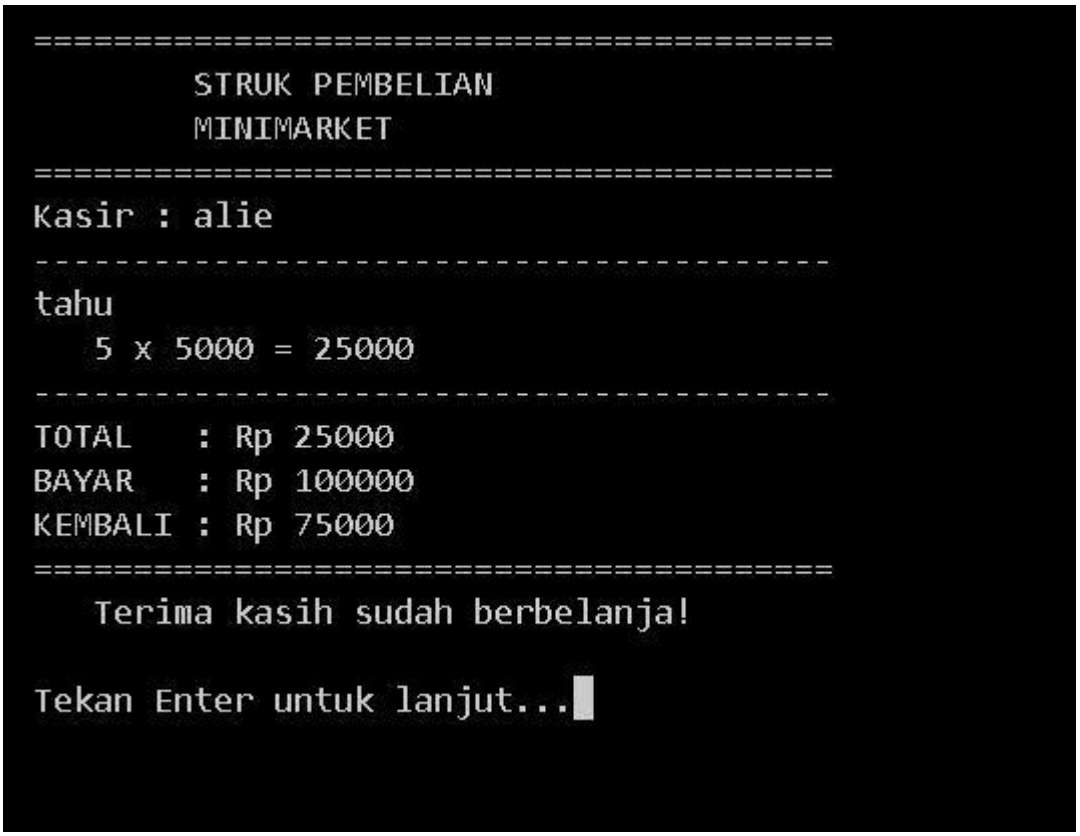
Gambar 4.11 Tampilan Tampilan User Tambah Ke Keranjang



Gambar 4.13 Tampilan Tampilan Hasil Lihat Keranjang



Gambar 4.14 Tampilan Tampilan Ubah Keranjang



Gambar 4.15 Tampilan Tampilan Hasil Cetak Struk

5. Langkah-langkah Git

5. 1 Git Add

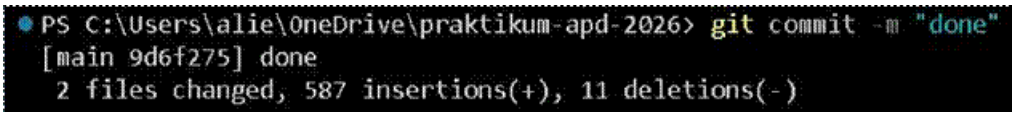
Perintah git add digunakan untuk menambahkan file baru atau perubahan file ke dalam staging area Git sebelum disimpan melalui commit. Perintah ini dapat digunakan untuk memilih file tertentu atau menambahkan semua perubahan.



***Gambar 5.1** Git Add*

5.2 Git Commit

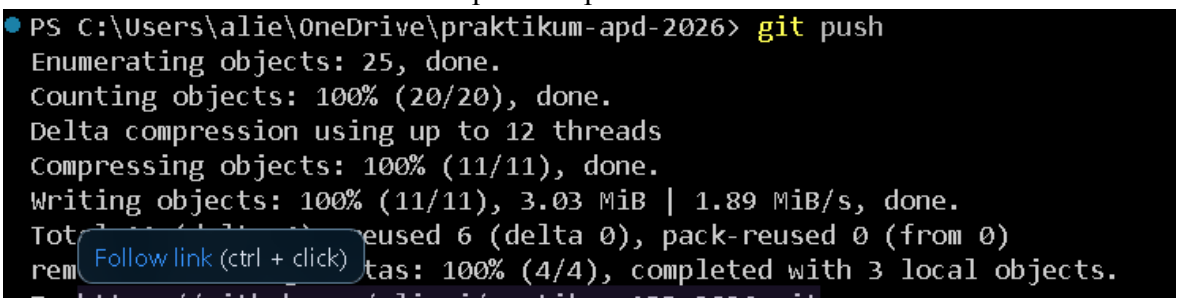
Perintah git commit digunakan untuk menyimpan perubahan yang sudah dimasukkan ke staging area sebagai sebuah riwayat perubahan di repositori lokal. Setiap commit dapat diberi pesan untuk menjelaskan perubahan yang dilakukan.



Gambar 5.2 *Git Commit*

5.3 Git Push

Perintah git push digunakan untuk mengirimkan commit dari repositori lokal ke repositori jarak jauh, misalnya GitHub. Dengan perintah ini, perubahan program dapat diunggah dan disimpan di repositori online.



Gambar 5.3 *Git Push*